



BugBoard26

Documentazione di Progetto

Corso di Ingegneria del Software 2025/2026

Gruppo di Lavoro:
Salvatore Indolino - N86004753

1 settembre 2026

Indice

Introduzione	3
1 Documento di Specifica dei Requisiti Software	4
1.1 Glossario	4
1.2 Individuazione e Caratterizzazione del Target degli Utenti	5
1.2.1 User Personas	5
1.3 Descrizione dei Requisiti Non-Funzionali e di Dominio	7
1.3.1 Requisiti Non-Funzionali	7
1.3.2 Requisiti di Dominio	8
1.4 Modellazione dei Casi d'Uso	9
1.4.1 Descrizione dettagliata dei casi d'uso:	9
1.5 Formalizzazione dei Casi d'Uso	11
1.5.1 Cockburn UC4 - Inserimento Nuova Issue	12
1.5.2 MockUp UC4	13
1.5.3 Cockburn UC5 - Consultazione e Filtraggio Board	14
1.5.4 MockUp UC5	15
2 Design del Sistema	16
2.1 Architettura e Criteri di Design	16
2.1.1 Server (Back-end)	16
2.1.2 Client (Front-end)	17
2.1.3 Modello logico per la persistenza dei dati	17
2.1.4 Scelte di Design dell'Interfaccia Utente	19
2.2 Scelte Tecnologiche e Sicurezza	20
3 Design del Software	22
3.1 Architettura a Livelli e Modello dei Dati	22
3.2 Pattern Applicativi e Dettagli Implementativi	23
3.2.1 Gestione dei Payload e DTO nel Back-end	23
3.2.2 Pattern di Interazione nel Front-end	23
3.3 Processo di Sviluppo, Versioning e Struttura del Progetto	24
3.3.1 Struttura del Livello Server (Back-end)	24
3.3.2 Struttura del Livello Client (Front-end)	25
3.4 Analisi Statica e Qualità del Codice	25
3.5 Containerizzazione e Build Automatica	26
3.5.1 Dockerfile: Immagine del Back-end	27
3.5.2 Orchestrazione e Build Automatica (Docker Compose)	27
3.5.3 Istruzioni di Deploy e Inizializzazione	28
3.5.4 Cloud Readiness e Predisposizione al Deployment	28

4	Attività di Testing e Valutazione dell'Usabilità	29
4.1	Strategia e Adozione dei Pattern xUnit	29
4.1.1	Ottimizzazione del Compilatore per i Test	29
4.2	Test Plan: Transizione di Stato delle Issue	29
4.3	Progettazione ed Esecuzione dei Test di Unità	30
4.3.1	Test Metodo 1: updateStatus (IssueService)	31
4.3.2	Test Metodo 2: assignUser (IssueService)	33
4.3.3	Test Metodo 3: addTag (IssueService)	34
4.4	Valutazione dell'Usabilità	35
4.4.1	Ispezione Sistematica basata su Checklist (Expert Review)	35
4.4.2	Test Empirico con Utenti Reali	35
4.4.3	Survey di Valutazione (Questionario)	36

Introduzione

BugBoard26 è una piattaforma software distribuita, finalizzata alla gestione collaborativa di issue in progetti software. L'obiettivo principale del sistema è fornire uno strumento robusto per inserire, tracciare e aggiornare lo stato di segnalazioni (come bug, funzionalità o problemi di documentazione) all'interno di un team di sviluppo. L'applicazione è composta da una Single Page Application (Front-end) sviluppata in React e da un server API RESTful (Back-end) basato su Node.js ed Express. Il sistema è progettato con una forte attenzione alla sicurezza, gestendo l'autenticazione in modo stateless tramite JSON Web Token (JWT) e differenziando le operazioni tramite un rigoroso Role-Based Access Control (RBAC) tra Amministratore e Membro del Team.

Struttura della Documentazione

Il seguente documento è suddiviso in quattro capitoli principali, strutturati come segue:

- **Documento di Specifica dei Requisiti Software:** In questa prima fase si definisce il glossario del progetto e si studiano a fondo i requisiti di sistema, sia architetturali che non funzionali (come l'affidabilità dell'input e la gestione sicura degli asset). Inoltre, vengono individuati e caratterizzati gli attori principali del sistema e viene formalizzato il modello dei casi d'uso.
- **Design del Sistema:** Questa sezione descrive le scelte architetture adottate, illustrando il pattern Client-Server, la suddivisione a livelli (Controller-Service-Repository) e le tecnologie di sicurezza selezionate (es. Zod, Multer) per garantire la robustezza dell'applicativo.
- **Design del Software:** Approfondisce le logiche implementative, partendo dal modello dei dati e dal ciclo di vita delle entità. Vengono inoltre analizzati i pattern architetturali del Front-end (es. Custom Hooks, rendering condizionale basato su RBAC), l'integrità transazionale del Back-end, i report di analisi statica del codice e il processo di containerizzazione tramite Docker.
- **Attività di Testing e Validazione:** L'ultima sezione spiega in che modo la logica di business del sistema è stata collaudata in isolamento, descrivendo la strategia di unit testing adottata tramite il framework Jest e la copertura delle classi di equivalenza.

Capitolo 1

Documento di Specifica dei Requisiti Software

1.1 Glossario

BugBoard26: Il nome della piattaforma software oggetto del presente documento, finalizzata alla gestione collaborativa di issue in progetti software.

Back-end: La componente lato server del sistema, sviluppata in Node.js/Express, responsabile della logica di business, dell'elaborazione delle API REST e della persistenza sicura dei dati sul database PostgreSQL.

Front-end: L'interfaccia utente (Client) della piattaforma, sviluppata in React. Permette l'interazione visuale con il sistema comunicando esclusivamente tramite API con il Back-end.

Amministratore (Admin): Utente del sistema con privilegi elevati. Oltre alle normali operazioni sulle issue, ha i permessi per creare nuove utenze, assegnare i bug ai membri del team, archiviare le issue e accedere ai report analitici.

Membro del Team: Utente base autenticato del sistema. Può creare nuove issue, commentarle, visualizzare quelle a lui assegnate e aggiornarne lo stato di avanzamento.

Issue: Termine generico che identifica una singola segnalazione all'interno della piattaforma. Può rappresentare un bug, una richiesta di funzionalità, un dubbio o un problema di documentazione.

Bug: Specifica tipologia di Issue utilizzata per segnalare un malfunzionamento, un errore o un comportamento anomalo del software.

Documentation: Tipologia di Issue utilizzata per segnalare errori, refusi o parti mancanti relativi alla documentazione del progetto.

Feature: Tipologia di Issue utilizzata per suggerire, richiedere o pianificare l'implementazione di nuove funzionalità all'interno del sistema.

Question: Tipologia di Issue utilizzata per richiedere chiarimenti tecnici o operativi ad altri membri del team.

Todo: Lo stato di default in cui si trova una issue nel momento esatto della sua creazione, indicante che l'attività deve ancora essere presa in carico.

In Lavorazione (In Progress): Stato operativo di una issue che indica l'avvenuta presa in carico da parte di un utente assegnatario.

Risolto (Resolved): Lo stato in cui viene portata una issue quando l'anomalia è stata corretta o la richiesta è stata soddisfatta. Il passaggio a questo stato innesca meccanismi automatici di notifica.

Archiviato (Archived): Stato definitivo che indica che una issue non è più visibile nelle liste attive, ma viene conservata nel database per ragioni di storicità e tracciabilità.

Notification (Notifica): Entità di sistema generata automaticamente in risposta ad eventi specifici (es. assegnazione di un bug), utile a mantenere aggiornati gli utenti.

JWT (JSON Web Token): Standard crittografico adottato per gestire in modo stateless l'autenticazione e l'autorizzazione degli utenti sulle rotte protette.

Swagger / OpenAPI: Strumento e standard utilizzati per la generazione della documentazione interattiva e testabile delle API REST esposte dal backend.

Tag (Etichetta): Stringa di testo personalizzabile che può essere associata ad una issue per facilitarne la ricerca, il filtraggio e la categorizzazione logica.

Zod: Libreria di validazione degli schemi dati utilizzata come middleware per garantire che gli input inviati dal client siano corretti prima dell'elaborazione.

UML Linguaggio visuale standard per progettare e documentare sistemi software.

Personas: Rappresentazioni archetipe degli utenti finali del sistema.

Mock-Up: Una rappresentazione visiva o prototipo di un'interfaccia utente o di una funzionalità di sistema

1.2 Individuazione e Caratterizzazione del Target degli Utenti

Il sistema si rivolge a un'utenza prettamente aziendale o di team di sviluppo software, suddivisa nei seguenti profili:

1.2.1 User Personas

Per modellare l'interfaccia e le funzionalità di BugBoard26 sulle reali esigenze degli utilizzatori finali, sono state elaborate due *User Personas* che rappresentano gli archetipi del nostro target di riferimento.

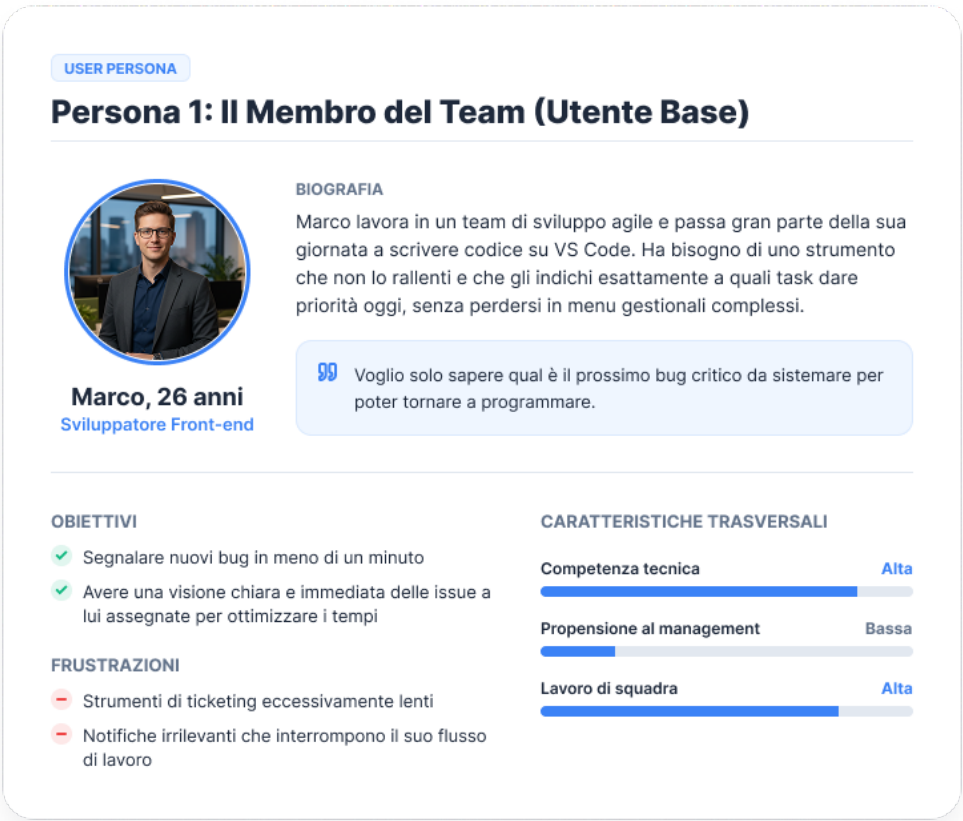


Figura 1.1: User Persona: Membro del Team (Utente Base)

Membro del Team (Utente Base)

Sviluppatori, tester, project manager o designer che lavorano operativamente sul progetto. Possiede un’elevata alfabetizzazione informatica ed è abituato a utilizzare strumenti digitali complessi e flussi di lavoro strutturati. Utilizza BugBoard26 quotidianamente e ha bisogno di un’interfaccia che permetta di inserire nuove segnalazioni nel minor tempo possibile e di consultare/aggiornare rapidamente lo stato dei bug a lui assegnati.

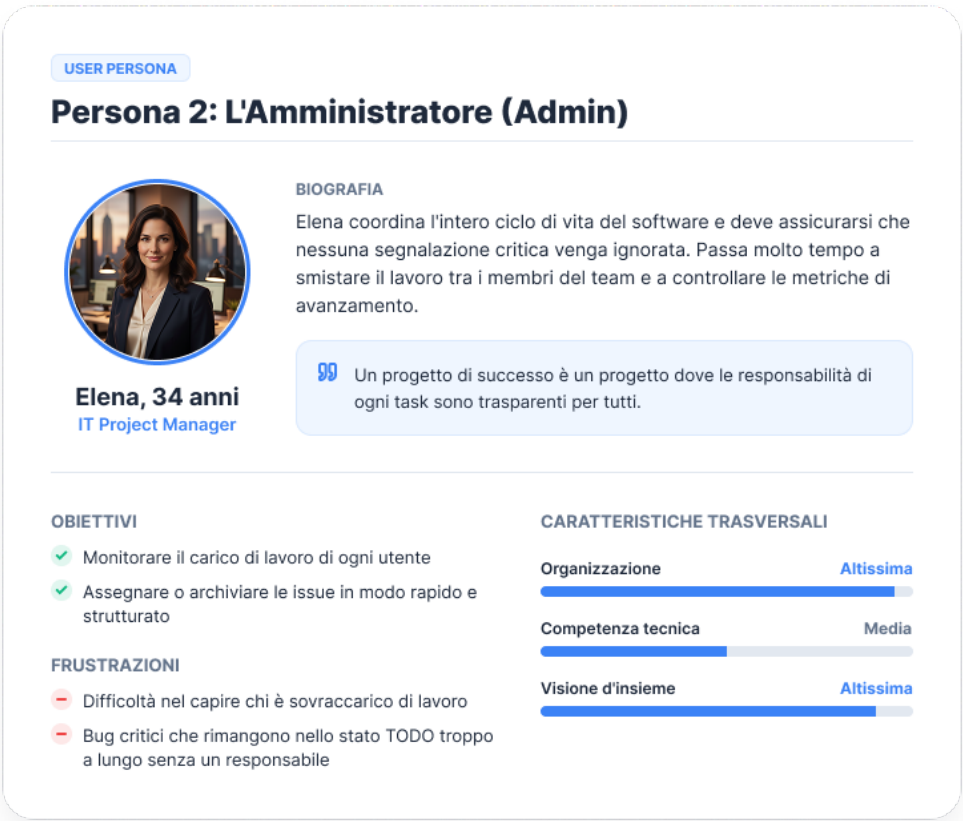


Figura 1.2: User Persona: Amministratore (Admin)

Amministratore (Admin)

Un utente privilegiato, tipicamente un Lead Developer, un Product Owner o un IT Manager. Possiede competenze tecniche elevate unite a una forte comprensione delle metriche di progetto e della gestione delle risorse umane. Oltre alle normali operazioni sui bug, l'Amministratore accede al sistema per configurare gli accessi e per monitorare l'andamento del lavoro tramite dashboard chiare e report mensili.

1.3 Descrizione dei Requisiti Non-Funzionali e di Dominio

In questa sezione vengono definiti i requisiti non-funzionali e di dominio per l'applicativo BugBoard26. I requisiti non-funzionali descrivono gli aspetti trasversali del sistema, garantendo sicurezza, performance e scalabilità. I requisiti di dominio riflettono invece le regole di business specifiche imposte dal committente per la gestione collaborativa delle issue.

1.3.1 Requisiti Non-Funzionali

- **Sicurezza e Integrità dei Dati:** Il sistema deve garantire la protezione delle rotte tramite un'architettura di autenticazione stateless basata su JSON Web Token (JWT). Le password devono essere protette tramite algoritmi di hashing irreversibile

(bcrypt). È richiesta un'implementazione rigorosa del *Role-Based Access Control* (RBAC) per limitare le azioni critiche ai soli Amministratori.

- **Affidabilità dell'Input e Prevenzione DoS:** Il sistema deve validare programmaticamente ogni payload in ingresso tramite la libreria *Zod*, respingendo richieste malformate prima dell'elaborazione logica. Il caricamento degli allegati deve essere protetto da middleware dedicati (*Multer*) con limiti hardware rigorosi (10MB) per prevenire attacchi *Denial of Service* legati all'esaurimento dello storage, e rinominare i file in ingresso tramite crittografia nativa (`crypto.randomUUID()`).
- **Architettura Distribuita (Separazione Client/Server):** L'applicativo deve implementare una rigida separazione tra front-end e back-end. Il back-end, sviluppato in Node.js, deve esporre API RESTful documentate interattivamente (Swagger) e operare in autonomia. Il front-end (Single Page Application in React) deve comunicare esclusivamente tramite rete, garantendo l'intercambiabilità dei moduli.
- **Usabilità, Reattività e Paradigmi di Sviluppo:** L'interfaccia deve garantire un'esperienza utente istantanea senza ricaricamenti della pagina, adattando dinamicamente la visualizzazione (Rendering Condizionale) in base ai privilegi dell'utente loggato. L'intero ecosistema deve essere sviluppato seguendo il paradigma Object-Oriented tramite TypeScript.
- **Manutenibilità e Portabilità:** Il sistema deve garantire la coerenza dei dati tramite transazioni ACID per le operazioni interdipendenti sul database relazionale. L'applicativo deve supportare la containerizzazione nativa (Docker) per garantire l'indipendenza dall'ambiente di esecuzione e la predisposizione al deploy in *Cloud*.

1.3.2 Requisiti di Dominio

- **Gestione Utenze e Gerarchia Aziendale:** Il sistema prevede due ruoli principali: Amministratore e Utente Normale (Membro del team). Solo un Amministratore ha i privilegi necessari per creare ulteriori account per i membri del team.
- **Struttura e Ciclo di Vita delle Issue:** Ogni segnalazione inserita nel sistema deve possedere obbligatoriamente un titolo e una descrizione. L'utente può specificare opzionalmente una priorità e allegare un'immagine. Le issue devono essere categorizzate in tipologie specifiche (*question*, *bug*, *documentation*, *feature*) e nascono di default nello stato iniziale "todo".
- **Regole di Assegnazione e Visibilità:** L'assegnazione di un task a un membro del team è un'operazione riservata esclusivamente all'Amministratore. Gli utenti normali possono modificare lo stato di avanzamento (es. verso "risolto") esclusivamente per i bug a loro direttamente assegnati, mentre gli Amministratori mantengono permessi di modifica globali.
- **Sistema di Notifiche:** Il flusso di lavoro deve essere supportato da notifiche automatiche. Il sistema deve generare un avviso sia quando un bug viene assegnato a un utente, sia quando quest'ultimo lo contrassegna come risolto (notificando l'autore originale della segnalazione).
- **Tracciabilità e Storicità (Audit Log):** Per ogni bug presente a sistema, deve essere mantenuta e resa consultabile una cronologia immutabile delle modifiche, indicando esplicitamente l'autore e la data del cambiamento (inclusa la creazione

iniziale). La rimozione di un bug dai flussi attivi (Archiviazione) è un'azione distruttiva permessa solo all'Amministratore, ma l'entità deve rimanere accessibile in una vista dedicata.

- **Dashboard e Reportistica Aziendale:** È richiesta un'area analitica riservata agli Amministratori che fornisca informazioni aggregate (es. bug aperti, assegnazioni per utente).

1.4 Modellazione dei Casi d'Uso

Di seguito il diagramma dei casi d'uso del sistema BugBoard26, che illustra le interazioni tra gli attori e le funzionalità principali.

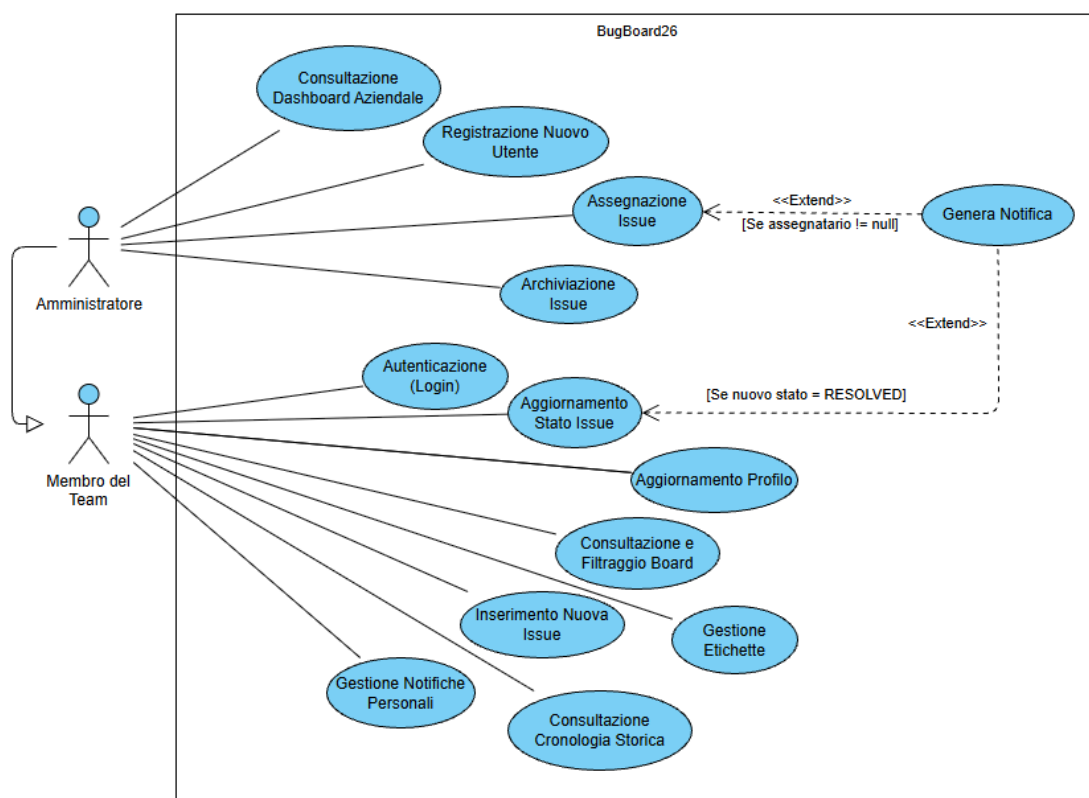


Figura 1.3: Diagramma dei Casi d'Uso di BugBoard26

1.4.1 Descrizione dettagliata dei casi d'uso:

- **UC1 - Autenticazione (Login):** L'utente inserisce le proprie credenziali per accedere. Il sistema valida i dati e restituisce un token JWT stateless per mantenere la sessione attiva in sicurezza. Si presuppone che tutti gli UC abbiano una relazione di *<< include >>* con l'Autenticazione.
- **UC2 - Registrazione Nuovo Utente:** Operazione riservata esclusivamente all'Amministratore, che compila i dati anagrafici e di ruolo per generare le credenziali di accesso di un nuovo membro del team.

- **UC3 - Aggiornamento Profilo:** L'utente autenticato modifica le proprie informazioni personali, caricando o aggiornando l'immagine del proprio Avatar tramite l'interfaccia.
- **UC4 - Inserimento Nuova Issue:** L'utente compila un form multimediale per creare un nuovo task, documentazione o bug, che entrerà nel sistema con lo stato iniziale "TODO".
- **UC5 - Consultazione e Filtraggio Board:** L'utente visualizza la lista delle segnalazioni attive, applicando filtri dinamici basati sullo stato di avanzamento o sull'utente assegnatario.
- **UC6 - Aggiornamento Stato Issue:** L'utente fa progredire un task (es. da IN PROGRESS a RESOLVED). La logica di dominio consente questa azione solo all'assegnatario del task o all'Amministratore.
- **UC7 - Assegnazione Issue:** L'Amministratore delega la risoluzione di una specifica problematica a un membro del team, innescando la generazione di una notifica automatica.
- **UC8 - Archiviazione Issue (Soft Delete):** L'Amministratore rimuove virtualmente una issue dalla visualizzazione principale transitandola nello stato "ARCHIVED", preservandone i dati per la reportistica futura.
- **UC9 - Gestione Etichette (Tag):** L'utente arricchisce la segnalazione associando o rimuovendo tag testuali. L'azione è limitata per sicurezza all'Amministratore o all'utente direttamente assegnato.
- **UC10 - Consultazione Cronologia Storica:** L'utente accede al registro immutabile (Audit Log) delle modifiche subite da una issue nel tempo, garantendo la tracciabilità assoluta di ogni cambio di stato o assegnazione.
- **UC11 - Gestione Notifiche Personali:** L'utente accede alla propria area privata per consultare gli avvisi di sistema (nuove assegnazioni o task risolti) e contrassegnarli come letti.
- **UC12 - Consultazione Dashboard Aziendale:** L'Amministratore visualizza un pannello aggregato che elabora metriche mensili, mostrando statistiche sul carico di lavoro degli utenti e sullo stato globale delle issue.

1.5 Formalizzazione dei Casi d'Uso

Al fine di descrivere con precisione il flusso di interazione tra l'attore e il sistema, in questa sezione vengono analizzati in maniera dettagliata due casi d'uso principali e non banali.

Per la stesura e la formalizzazione è stato adottato lo stile strutturato proposto da Alistair Cockburn nel suo celebre testo *Writing Effective Use Cases*. Il template tabellare è stato riadattato per il contesto del progetto BugBoard26 e permette di evidenziare in modo chiaro e schematico lo scopo, le precondizioni, lo scenario principale di successo e le relative estensioni (ovvero la gestione delle anomalie e i percorsi alternativi).

I casi d'uso selezionati per questa analisi di dettaglio sono **UC4 - Inserimento Nuova Issue** e **UC5 - Consultazione e Filtraggio Board**.

1.5.1 Cockburn UC4 - Inserimento Nuova Issue

Nome Caso d'Uso	UC4 - Inserimento Nuova Issue
Attore Principale	Utente Autenticato (Amministratore o Membro del Team)
Scopo	L'Utente vuole creare una nuova Issue
Breve Descrizione	L'utente compila e invia un modulo per segnalare un nuovo problema, richiedere una funzionalità o chiedere chiarimenti.
Precondizioni	L'utente ha effettuato con successo il login al sistema (token JWT valido) e si trova nella dashboard principale.
Postcondizioni	Il sistema ha salvato correttamente la nuova issue nel database, assegnandole di default lo stato "TODO".
Scenario Principale	1. L'utente seleziona l'opzione per creare una nuova issue. 2. Il sistema presenta il modulo di inserimento vuoto. 3. L'utente compila i campi "Titolo" e "Descrizione". 4. L'utente seleziona la "Tipologia". 5. (Opzionale) L'utente seleziona la priorità (LOW, MEDIUM o HIGH) e allega un'immagine. 6. L'utente conferma l'invio. 7. Il middleware del sistema (Zod/Multer) valida programmaticamente i dati ricevuti (es. controllo campi minimi e grandezza file). 8. Il sistema persiste la issue nel database e restituisce un codice HTTP 201 (Created). 9. Il sistema mostra un messaggio di conferma e reindirizza l'utente alla vista riepilogativa.
Estensioni	7a. Dati mancanti o malformati: Il middleware Zod intercetta l'anomalia sui campi obbligatori o sui formati e solleva un'eccezione bloccante. Il sistema restituisce HTTP 400 elencando i campi errati. L'interfaccia attende la correzione dell'utente. 7b. Formato o dimensione immagine non validi: Il middleware di upload blocca il file se supera i 5 10MB consentiti, restituendo un errore specifico. L'utente è invitato a scegliere un file idoneo.

1.5.2 MockUp UC4

The image shows a browser window titled "Browser" with a standard address bar and navigation buttons. Inside the browser, there is a form titled "CREA UNA NUOVA ISSUE" with a close button (X) in the top right corner. The form contains the following fields and controls:

- Titolo ***: A single-line text input field.
- Descrizione ***: A multi-line text area.
- Tipologia**: A dropdown menu with "Bug" selected.
- Priorità**: A dropdown menu with "Alta" selected.
- Allegato (Immagine)**: A large rectangular area with a camera icon and the text "Trascina qui un immagine o clicca per caricare".
- Buttons**: "Annulla" (Cancel) and "Crea Issue" (Create Issue) buttons at the bottom right.

Figura 1.4: Mock-up UI: Creazione Nuova Issue

1.5.3 Cockburn UC5 - Consultazione e Filtraggio Board

Nome Caso d'Uso	UC5 - Consultazione e Filtraggio Board
Attore Principale	Utente Autenticato (Amministratore o Membro del Team)
Scopo	L'Utente vuole visualizzare la lista delle Issue
Breve Descrizione	L'utente visualizza l'elenco delle issue, potendo applicare filtri di ricerca e criteri di ordinamento.
Precondizioni	L'utente ha effettuato con successo il login ed è in possesso di un token di sessione valido.
Postcondizioni	Il sistema mostra l'elenco delle issue aggiornate che rispettano i criteri di ricerca impostati (es. filtraggio per Stato o per Assegnatario).
Scenario Principale	1. L'utente naviga verso la Dashboard della lista issue. 2. Il sistema recupera l'elenco delle issue salvate e le presenta in ordine di default. 3. L'utente seleziona i parametri di filtro (Stato, Assegnatario) e avvia la ricerca tramite Query String. 4. Il sistema elabora la richiesta recuperando le issue corrispondenti dal database relazionale. 5. Il sistema aggiorna dinamicamente la visualizzazione mostrando i risultati filtrati.
Estensioni	2a. Nessuna issue presente: Il sistema mostra il messaggio "Nessuna issue trovata". 4a. Nessun risultato per i filtri applicati: Il sistema avvisa l'utente con il messaggio "Nessuna issue trovata" e fornisce un pulsante o un controllo visivo per resettare i filtri di ricerca.

1.5.4 MockUp UC5

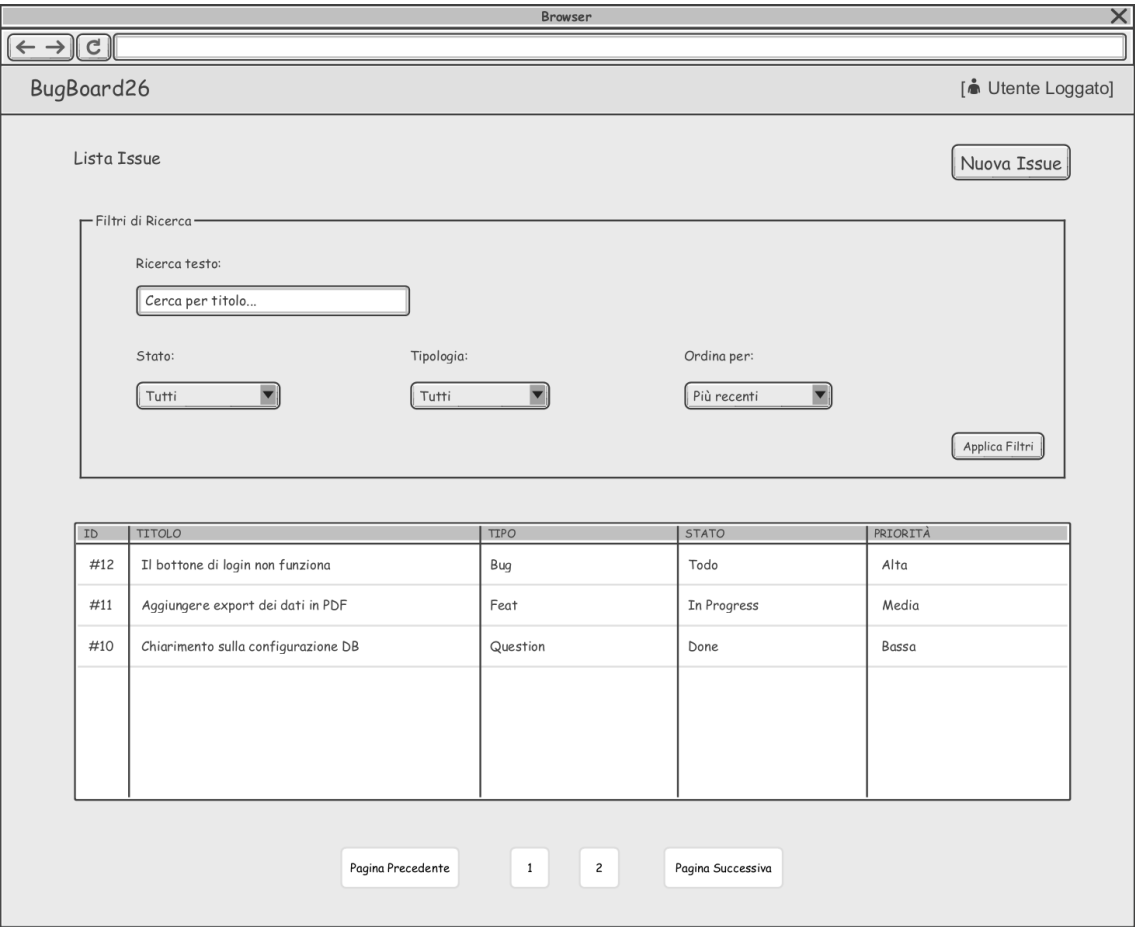


Figura 1.5: Mock-up UI: Lista Issue con filtri

Capitolo 2

Design del Sistema

2.1 Architettura e Criteri di Design

Il sistema BugBoard26 adotta un'architettura distribuita di tipo Client-Server, basata sull'interazione tra una Single Page Application (Front-end) e un server API RESTful (Back-end). Questa separazione garantisce l'assoluta indipendenza dei macro-componenti e ne facilita il ciclo di vita, rispondendo in pieno ai vincoli di progetto.

2.1.1 Server (Back-end)

Il modulo Server del sistema BugBoard26 è stato implementato in ambiente Node.js avvalendosi del framework Express ed è scritto interamente in TypeScript. Agisce come provider centralizzato di API RESTful, gestendo la persistenza dei dati e la logica di business in modo totalmente slegato e indipendente dal livello di presentazione client.

Per garantire massima manutenibilità, scalabilità e testabilità, l'architettura del Back-end adotta un rigoroso **Pattern a Livelli (Layered Architecture)**, strutturato specificamente sul modello Controller-Service-Repository:

- **Presentation Layer (Controller):** Costituisce il punto di ingresso esclusivo per tutte le richieste di rete. I Controller hanno la singola responsabilità di intercettare il traffico HTTP, estrarre i parametri o i payload e formattare le risposte JSON con i relativi *status code* HTTP (es. 200 OK, 201 Created). Essi delegano totalmente l'elaborazione dei dati al livello sottostante, fungendo unicamente da interfacce di smistamento.
- **Business Logic Layer (Service):** Incapsula il cuore pulsante (core domain) dell'applicazione. I Service ricevono i dati validati dai Controller, applicano le regole applicative (come la logica di transizione di stato di una Issue o l'innescio di una notifica a seguito di un'assegnazione) e orchestrano le chiamate al database. Questo livello di astrazione rende la logica di dominio facilmente testabile in totale isolamento tramite framework xUnit.
- **Data Access Layer (Prisma ORM):** Astrazione del livello di persistenza che sostituisce il tradizionale approccio a query SQL testuali fornendo un'interfaccia a oggetti fortemente tipizzata (Type-Safe). Prisma si fa carico di interagire in sicurezza con il database relazionale PostgreSQL, prevenendo strutturalmente vulnerabilità gravissime come le SQL Injection e garantendo la mappatura esatta tra le entità relazionali e le interfacce TypeScript.

- **Integrità Transazionale (ACID):** Per gestire operazioni di dominio interdipendenti, il back-end fa uso di transazioni atomiche. Se la chiusura di un bug, la creazione del log storico (*HistoryLog*) e la generazione della notifica dovessero subire un'interruzione di sistema a metà processo, l'operatore `prisma.$transaction` garantisce un *rollback* completo, impedendo la generazione di dati fantasma e preservando la coerenza assoluta del database.

2.1.2 Client (Front-end)

Il modulo Client del sistema BugBoard26 è stato sviluppato come una Single Page Application (SPA) avvalendosi della libreria React. Questa scelta architetturale è dettata dalla necessità di fornire un'esperienza utente nativa e interattiva: le transizioni tra le viste e gli aggiornamenti dei dati avvengono dinamicamente manipolando il DOM, senza la necessità di ricaricare l'intera pagina web dal server, riducendo drasticamente il carico sulla rete e i tempi di latenza.

A livello di design, lo sviluppo del Front-end abbandona i pattern monolitici in favore di un **Component-Based Pattern** integrato con l'uso di **Custom Hooks**. Questa precisa organizzazione del codice garantisce molteplici vantaggi ingegneristici:

- **Separation of Concerns:** L'interfaccia visiva è stateless e nettamente separata dalla logica di business. L'intera gestione degli stati, del filtraggio dati e della comunicazione HTTP è delegata e incapsulata all'interno di Custom Hooks dedicati, rendendo il codice altamente modulare e predisposto al testing isolato.
- **Rendering Condizionale basato sui Ruoli (RBAC):** L'interfaccia non è statica, ma si adatta dinamicamente al livello di autorizzazione dell'utente. Leggendo le direttive del token di sessione, i componenti React valutano in tempo reale se renderizzare o nascondere specifiche *Call to Action* critiche (come i pulsanti per l'assegnazione o l'archiviazione di una issue), abilitandole esclusivamente se l'utente possiede i privilegi di Amministratore.
- **Centralizzazione del Networking:** Il livello di comunicazione con le API REST è centralizzato tramite un'istanza dedicata della libreria Axios. Sfruttando il pattern architetturale dei *Request Interceptors*, il Client inietta in modo trasparente e automatico il token JWT nell'header di ogni richiesta in uscita, garantendo la sicurezza delle chiamate senza duplicare codice nei singoli componenti visivi.
- **Reattività Strutturale:** Grazie al Virtual DOM di React, l'applicativo reagisce istantaneamente ai cambiamenti di stato. Quando un utente aggiorna lo stato di una issue, il framework ricalcola e ri-renderizza esclusivamente il frammento di interfaccia interessato dalla modifica, garantendo performance ottimali anche in presenza di board ricche di elementi.

2.1.3 Modello logico per la persistenza dei dati

Per garantire una mappatura chiara tra il dominio applicativo e il database relazionale PostgreSQL, di seguito viene descritto il modello logico delle entità, esplicitando gli attributi principali e le relative chiavi primarie (PK) e secondarie (FK).

Entità	Descrizione e Attributi
User	Rappresenta un utente del sistema (Membro del Team o Amministratore). id: int [PK] - Identificatore univoco dell'utente. email: String - Indirizzo email, utilizzato per l'autenticazione. password: String - Hash crittografico della password. role: Enum - Ruolo per il controllo degli accessi (es. ADMIN, MEMBER). fullName: String - Nome completo dell'utente. avatarUrl: String - Percorso per l'immagine del profilo. createdAt: DateTime - Marca temporale di registrazione.
Issue	Il nucleo centrale del sistema, rappresenta una singola segnalazione. id: int [PK] - Identificatore univoco del bug o task. title: String - Titolo riassuntivo. description: String - Dettaglio esteso della segnalazione. status: Enum - Stato di avanzamento (TODO, IN PROGRESS, RESOLVED, ARCHIVED). type: Enum - Categoria strutturale (Bug, Feature, Question, Documentation). priority: Enum - Livello di criticità assegnato (Low, Medium, High). createdAt / updatedAt / resolvedAt: DateTime - Timestamp del ciclo di vita. assigneeId: int [FK] - Riferimento all'utente incaricato della risoluzione. reporterId: int [FK] - Riferimento all'utente creatore della segnalazione.
Tag	Etichetta semantica per la categorizzazione rapida. id: int [PK] - Identificatore univoco dell'etichetta. name: String - Testo identificativo.
HistoryLog	Traccia immutabile della cronologia delle modifiche per l'audit trail. id: int [PK] - Identificatore univoco della riga di log. action: String - Tipologia di modifica effettuata (es. STATUS_UPDATE). oldValue / newValue: String - Fotografia dei valori precedenti e successivi all'azione. modifiedAt: DateTime - Marca temporale in cui è avvenuta la modifica. issueId: int [FK] - Riferimento alla Issue che ha subito la modifica. modifierId: int [FK] - Riferimento all'utente che ha eseguito l'operazione.
Notification	Avviso generato automaticamente dal sistema in risposta agli eventi. id: int [PK] - Identificatore univoco della notifica. message: String - Testo descrittivo dell'avviso. isRead: Boolean - Flag di visualizzazione (letto / non letto). createdAt: DateTime - Marca temporale di generazione dell'avviso. userId: int [FK] - Riferimento all'utente destinatario della notifica.

Tabella 2.1: Dizionario dei dati e attributi di persistenza

2.1.4 Scelte di Design dell’Interfaccia Utente

L’interfaccia utente di BugBoard26 è stata progettata con l’obiettivo di garantire un’esperienza fluida, intuitiva e priva di distrazioni, ottimizzando i tempi di interazione per i team di sviluppo. Le scelte di design si basano su principi ergonomici, gerarchia visiva e feedback immediato.

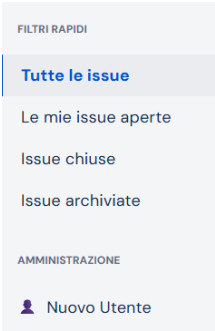
Anatomia dell’Applicazione

L’interfaccia si sviluppa su un layout a macro-aree strutturali (come le viste principali Login, Board e Dashboard) per facilitare l’orientamento, rispettando il principio gestaltico della "buona forma":

- **TopNav (Barra Superiore):** Riservata alla navigazione verso le viste principali (come la Dashboard), alla consultazione delle notifiche asincrone di sistema e all’accesso al menu del profilo utente.



- **Sidebar (Navigazione Laterale):** Ospita i filtri veloci per raffinare in tempo reale la visualizzazione delle segnalazioni e, limitatamente agli utenti con privilegi amministrativi, l’interfaccia dedicata alla registrazione di nuovi membri del team.



- **Body (Area di Lavoro):** Lo spazio centrale, che sfrutta tutta la larghezza disponibile rimuovendo i limiti di massima larghezza e i margini automatici, dedicato alla visualizzazione e interazione diretta con le segnalazioni.

Issues

Create

Cerca per titolo...Tag...Status: TuttiTipo: TuttiPiù recenti (1)CercaReset

KEY	SUMMARY	TYPE	STATUS	PRIORITY	REPORTER	ASSIGNEE
ID-5	Bottone login non funzionante	BUG	TODO		Salvatore Indolino	System Administrator
ID-4	Aggiornare diagramma UML del database	DOCUMENTATION	TODO		D. dev	Unassigned
ID-3	Implementare drag-and-drop sulla board	FEATURE	TODO		D. dev	T. tes
ID-2	Crash del server con token malformato	BUG	RESOLVED		D. dev	T. tes
ID-1	Errore caricamento pagina principale	BUG	TODO		Salvatore Indolino	T. tes

Principi di Usabilità e Legge di Fitts

In conformità con la Legge di Fitts, le *Call to Action* (CTA) primarie sono posizionate

in aree strategiche e dotate di una superficie cliccabile adeguata per minimizzare i tempi di puntamento. Le interfacce a comparsa isolano il focus dell'utente sull'azione corrente, riducendo drasticamente il carico cognitivo.

Principio di Vicinanza (Gestalt)

Per l'organizzazione visiva dei metadati dei bug, è stato applicato il principio della vicinanza. Le informazioni correlate sono raggruppate in cluster visivi ravvicinati, permettendo all'utente di processare lo stato di un task con un singolo colpo d'occhio.

CSS Nativo, Design System e Variabili Globali

Per la realizzazione della UI, invece di affidarsi a framework esterni, si è optato per un approccio robusto basato su CSS nativo e variabili CSS (Custom Properties):

- **Design System Centralizzato:** Il progetto adotta una palette cromatica in stile *Jira*, definita a livello globale nel file `index.css` attraverso variabili come `--bg` (sfondo generale), `--surface` (sfondi di card e modali), `--text-h` (testo primario) e `--accent` (colore primario).
- **Single Source of Truth:** Oltre all'utilizzo nei fogli di stile, i token cromatici sono stati tipizzati e centralizzati nel file `theme.ts` all'interno dell'oggetto esportato `UI_COLORS` (es. `background: '#F4F5F7', primary: '#0052CC', textPrimary: '#172B4D'`). Questo garantisce assoluta coerenza visiva permettendo l'uso programmatico dei colori all'interno della logica React o di eventuali grafici.
- **Coerenza Tipografica e Accessibilità:** La gerarchia tipografica è gestita uniformemente attraverso il font *DM Sans*, configurato come `--sans` e `--heading`. L'usabilità è stata curata anche nei dettagli tecnici, come l'implementazione di `hack CSS` specifici (`-webkit-box-shadow: 0 0 0 1000px var(--surface) inset`) per sovrascrivere gli stili di autofill predefiniti dei browser, mantenendo inalterata l'esperienza cromatica dell'utente.

2.2 Scelte Tecnologiche e Sicurezza

Lo stack tecnologico selezionato garantisce robustezza, manutenibilità e la totale aderenza ai vincoli orientati agli oggetti. Un'attenzione particolare è stata dedicata alla sicurezza (*Security by Design*) e alla prevenzione di attacchi web comuni.

- **TypeScript ed Ecosistema Node.js:** Applicato in logica full-stack per sfruttare la tipizzazione statica avanzata. Il back-end si affida al framework Express, potenziato da policy CORS (*Cross-Origin Resource Sharing*) limitate esclusivamente ai domini autorizzati, neutralizzando accessi malevoli da origin di terze parti.
- **Autenticazione Stateless e RBAC:** La gestione degli accessi è demandata allo standard JSON Web Token (JWT). L'adozione di middleware custom per il *Role-Based Access Control* (RBAC) garantisce la protezione granulare degli endpoint REST, distinguendo tra utenza standard e privilegi amministrativi.
- **Validazione a Runtime (Zod):** Per garantire l'affidabilità dell'input client, è stata adottata la libreria Zod. Attraverso un middleware dedicato, i payload in ingresso vengono validati contro schemi rigorosi: le richieste malformate vengono scartate ancor prima di raggiungere i Controller, azzerando i rischi legati a dati inconsistenti.

- **PostgreSQL e Prisma ORM:** Oltre a garantire la type-safety, l'utilizzo dell'ORM funge intrinsecamente da livello di sanitizzazione continuo per le interazioni con il database. Questo approccio previene strutturalmente le vulnerabilità critiche come le *SQL Injection*.
- **Gestione Sicura degli Asset (Multer):** Il caricamento degli allegati alle issue è gestito tramite middleware dedicato. Per scongiurare attacchi di tipo *Denial of Service* (DoS), è stato implementato un blocco rigoroso a livello di buffer, impostando un limite di 10MB per file. Inoltre, i file vengono rinominati utilizzando generatori pseudo-casuali crittograficamente sicuri (`crypto.randomUUID()`), mitigando le vulnerabilità da Weak Cryptography.
- **React (Prevenzione XSS):** Sul fronte Client, oltre ai vantaggi prestazionali, React esegue intrinsecamente l'escape dei valori inseriti nelle viste (JSX). Questo fornisce al sistema una forte protezione di base contro gli attacchi di tipo *Cross-Site Scripting* (XSS), impedendo l'esecuzione di script malevoli iniettati tramite input utente.
- **Docker:** Il sistema è stato interamente predisposto per la containerizzazione, permettendo di pacchettizzare Back-end e Database in ambienti isolati, annullando i conflitti di dipendenza ed evidenziando una spiccata propensione verso le moderne metodologie DevOps.

Capitolo 3

Design del Software

3.1 Architettura a Livelli e Modello dei Dati

Come illustrato nel Diagramma delle Classi (Figura 3.1), il back-end è strutturato su un'architettura a tre livelli: i **Controllers** gestiscono il traffico HTTP, i **Services** incapsulano la logica di business e le **Entities** rappresentano il modello fisico del database.

Lo schema dati relazionale è modellato nel rigoroso rispetto dell'integrità del dominio:

- **User:** Gestisce l'autenticazione. Possiede una relazione 1-a-molti con l'entità Issue (assumendo il ruolo di autore o di assegnatario) e con l'entità HistoryLog.
- **Issue:** Il nucleo centrale del sistema. Intrattiene una relazione di composizione stretta con *HistoryLog*. Tale vincolo forte garantisce che l'intero audit trail (la cronologia delle modifiche) segua in modo indissolubile il ciclo di vita del bug segnalato.
- **Tag:** Modella la categorizzazione semantica, relazionata tramite cardinalità multi-a-molti (N:M) con l'entità Issue.
- **HistoryLog:** Traccia in modo immutabile l'audit trail di ogni singola Issue, registrando vecchi e nuovi valori ad ogni transizione di stato.
- **Notification:** Gestisce gli avvisi asincroni generati dal sistema (es. innescati dalle assegnazioni), mantenendo il riferimento all'utente destinatario.

Livelli di Presentazione e Logica di Business

A completamento del modello dei dati, il diagramma definisce le classi responsabili della gestione del traffico HTTP e dell'applicazione delle regole di dominio:

- **Controllers (UserController, IssueController, ReportController, NotificationController):** Rappresentano i punti di ingresso delle API REST. Si occupano di intercettare le richieste, estrarre i payload già validati dai middleware (come Zod) e formattare le risposte HTTP, delegando l'elaborazione ai rispettivi Service.
- **Services (UserService, IssueService, ReportService, NotificationService):** Contengono la vera *business logic* dell'applicativo. Espongono metodi asincroni che implementano i casi d'uso (es. `updateStatus`, `assignUser`, `getDashboardMetrics`) e orchestrano le chiamate al database tramite l'istanza centralizzata `PrismaClient`.

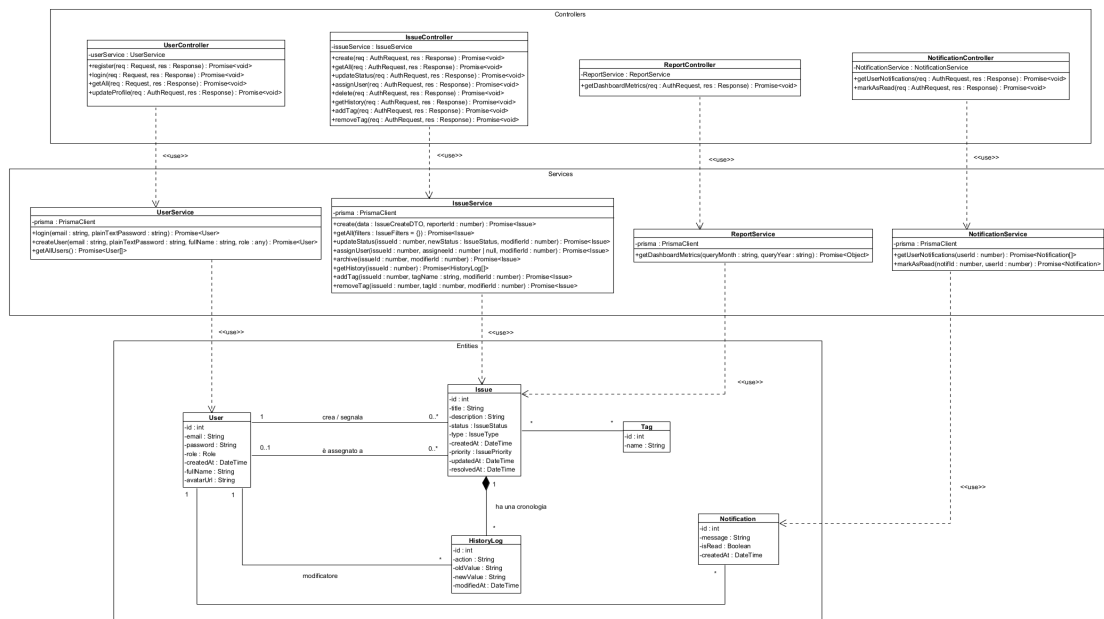


Figura 3.1: Diagramma delle Classi - Livelli Controller, Service, Entities (Clicca sull'immagine per ingrandire)

3.2 Pattern Applicativi e Dettagli Implementativi

Mentre l'architettura di sistema definisce la struttura globale, a livello di codice (Design del Software) sono stati applicati pattern specifici per risolvere problematiche puntuali di UI e gestione dei dati.

3.2.1 Gestione dei Payload e DTO nel Back-end

- **Sicurezza e Serializzazione:** A livello di Service, le entità recuperate dal database non vengono mai restituite direttamente al client. Prima della serializzazione JSON, i dati vengono filtrati applicando il pattern DTO (*Data Transfer Object*) per rimuovere programmaticamente campi sensibili (come la password, precedentemente sottoposta a hashing irreversibile tramite `bcrypt` con 10 salt rounds). Questo garantisce che non vi siano *data leak* nelle risposte HTTP.
- **Atomicità Operativa:** Come definito nei requisiti di architettura, le transazioni garantiscono l'integrità dei dati. A livello di codice, questo si traduce nell'uso intensivo del blocco `prisma.$transaction`, all'interno del quale vengono accodate le *Promise* di update della Issue, insert dell'*HistoryLog* e insert della Notifica, eseguendole come una singola unità logica isolata.

3.2.2 Pattern di Interazione nel Front-end

Nel livello di presentazione in React, le dinamiche di interazione avanzate sono gestite tramite design pattern specifici:

- **Isolamento degli Stati Complessi:** Il componente principale *Board* opera come pura interfaccia di rendering (*Presentational Component*). La mole di logica com-

plexa riguardante la gestione degli stati di ricerca, il filtraggio e la paginazione è interamente incapsulata in un custom hook dedicato (`useBoardData`), mantenendo il componente visivo snello e focalizzato esclusivamente sulla UI.

- **Click-Outside Pattern:** Per la gestione fluida delle interfacce a comparsa (come le modali dei dettagli bug o i menu dropdown), il sistema implementa un pattern di intercettazione globale. Combinando l'hook `useRef` con l'aggiunta di *event listener* sull'oggetto `window`, il componente rileva i click esterni al proprio perimetro, innescando l'auto-chiusura e gestendo in sicurezza la pulizia degli eventi in fase di smontaggio (*cleanup function*) per evitare *memory leak*.
- **Sincronizzazione Asincrona (Polling):** Per mantenere l'utente aggiornato sullo stato delle proprie segnalazioni senza ricorrere a connessioni persistenti gravose come i WebSocket, il layer client implementa un meccanismo di *Long Polling*. Il sistema effettua verifiche temporizzate e trasparenti ogni 10 secondi, recuperando le nuove notifiche in background e aggiornando i contatori visivi in tempo reale.

3.3 Processo di Sviluppo, Versioning e Struttura del Progetto

Lo sviluppo è stato condotto applicando metodologie agili e pratiche di Software Engineering moderne. Il controllo di versione è stato gestito integralmente tramite Git, permettendo un tracciamento granulare del lavoro collaborativo e la gestione sicura delle modifiche. L'intera repository, comprensiva degli artefatti e del file di configurazione Docker, è accessibile e verificabile su GitHub: [Repository BugBoard26](#).

Al fine di mantenere una netta *Separation of Concerns* pur gestendo l'intero applicativo in un unico ecosistema, il progetto è stato organizzato in due macro-ambienti logicamente e fisicamente isolati, orchestrati dal file `docker-compose.yml` presente nella *root* principale.

3.3.1 Struttura del Livello Server (Back-end)

Il codice sorgente del back-end segue una rigida gerarchia a livelli per isolare le responsabilità e facilitare la manutenzione:

- `/prisma`: Contiene il file `schema.prisma`, che funge da *Single Source of Truth* per la modellazione dichiarativa del database, e gli script di *seed* per l'inizializzazione del sistema (es. creazione dell'utente Admin).
- `/src/routes`: Definisce gli endpoint esposti dall'API REST (es. `issueRoutes.ts`), associando ad ogni rotta la sua catena di middleware e il relativo controller.
- `/src/middlewares`: Contiene le funzioni di intercettazione e controllo, come `authMiddleware` per la verifica del token JWT, `uploadMiddleware` per l'intercettazione dei file tramite Multer e i validatori dell'input.
- `/src/schemas`: Raccoglie i modelli di validazione dichiarativa scritti con **Zod** per blindare le richieste HTTP in ingresso.
- `/src/controllers`: Moduli del *Presentation Layer* incaricati di gestire esclusivamente l'I/O HTTP, estraendo i payload e formattando le risposte JSON.

- `/src/services`: Il *core* dell'applicativo. Contiene le classi della logica di business (es. `IssueService`) che applicano le regole di dominio e orchestrano le transazioni sul database.
- `/src/__tests__`: Isola le *test suite* scritte in Jest, garantendo la verifica automatizzata dei Service.

3.3.2 Struttura del Livello Client (Front-end)

Il codice sorgente dell'interfaccia React, inizializzata tramite il bundler *Vite*, è stato organizzato per favorire la riusabilità, separando nettamente le viste, i componenti e la logica di rete:

- `/src/components`: Contiene i blocchi visivi riutilizzabili dell'interfaccia utente (*Presentational Components*). Include le card delle issue (`IssueCard.tsx`), gli elementi di layout (`Sidebar.tsx`, `TopNav.tsx`) e i moduli a comparsa (`CreateIssueModal.tsx`, `IssueDetailModal.tsx`).
- `/src/pages`: Raccoglie i componenti contenitore (*Container Components*) che rappresentano le macro-viste dell'applicativo (`Login.tsx`, `Board.tsx` e `Dashboard.tsx`).
- `/src/hooks`: Incapsula la logica di stato, il recupero dei dati e le logiche di manipolazione tramite *Custom Hooks* (es. `useBoardData.ts`), disaccoppiando il comportamento del sistema dal rendering visivo.
- `/src/services`: Delega e centralizza il livello di networking. Oltre alla configurazione di base di *Axios* (`api.ts`), isola le chiamate REST in moduli specifici per dominio (es. `authService.ts`, `issueService.ts`, `userService.ts`).
- `/src/styles` e `/src/types`: Definiscono rispettivamente il tema grafico globale dell'applicativo (`theme.ts`) e le definizioni delle interfacce TypeScript (`index.ts`) per garantire la *type-safety* fino al livello client.

3.4 Analisi Statica e Qualità del Codice

La qualità del codice è un aspetto fondamentale nello sviluppo software. Un codice di alta qualità non solo facilita la manutenzione e l'evoluzione del sistema, ma contribuisce anche alla sua affidabilità, efficienza e sicurezza. In conformità con le specifiche di progetto, per garantire l'eccellenza qualitativa dell'applicativo e validare oggettivamente le scelte architettureali, la *codebase* del **Back-end** è stata sottoposta ad analisi statica continua tramite l'infrastruttura **SonarQube**, eseguita in un ambiente containerizzato dedicato.

Come documentato in Figura 3.2, il refactoring costante ha permesso di azzerare il debito tecnico (*Technical Debt*) e di raggiungere i massimi punteggi (*Grado A*) su tutti gli assi di valutazione:

- **Sicurezza (Security)**: Assenza totale di vulnerabilità (0 Vulnerabilities). Durante la fase di sviluppo, gli analizzatori euristici hanno segnalato dei *Security Hotspots* preventivi relativi alla gestione dei file in ingresso e al protocollo CORS. Tali hotspot sono stati risolti implementando controlli crittografici nativi (`crypto`) e validatori di dimensione hardware sul middleware HTTP, portando il livello di sicurezza al 100%.

- **Affidabilità (Reliability):** Il report evidenzia l'assenza di bug (0 Bugs). L'uso massiccio di controlli sui tipi in TypeScript (*Type Guards*) e la rigorosa astrazione a livelli hanno impedito strutturalmente l'insorgenza di bug logici e *Null Pointer Exception*.
- **Manutenibilità (Maintainability) e Code Smells:** Il rispetto dei principi SOLID (in particolare della Singola Responsabilità) ha ridotto al minimo la complessità ciclomatica delle funzioni. L'analisi non rileva alcun *Code Smell*, garantendo un codice pulito e altamente estensibile.
- **Indice di Duplicazione (Duplications):** L'analisi evidenzia uno **0.0% di righe duplicate**. Questo dato certifica l'applicazione rigorosa del principio DRY (*Don't Repeat Yourself*), ottenuto tramite l'uso metodico di funzioni di *utility*, *custom middleware* e astrazioni efficaci a livello di Service.

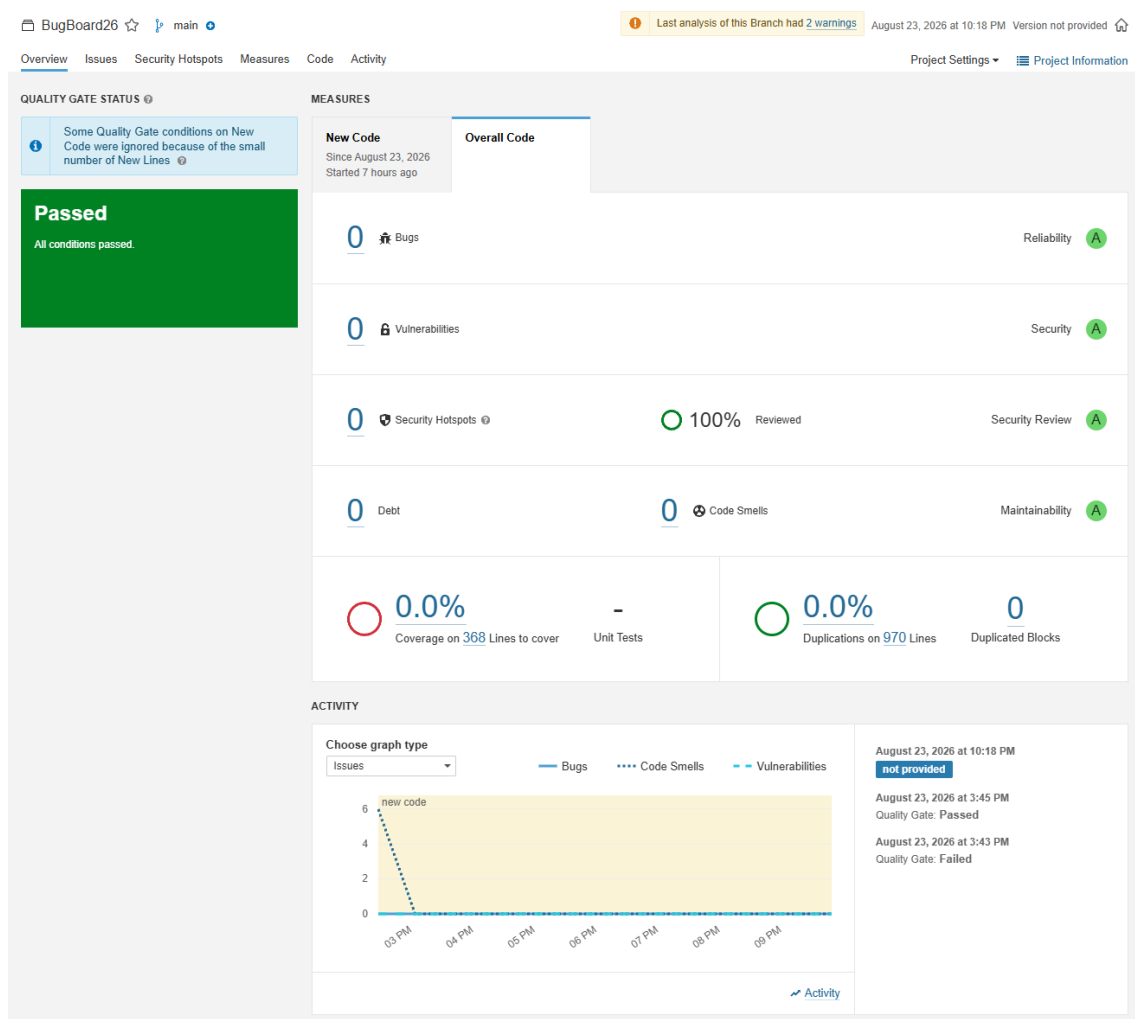


Figura 3.2: Report di Qualità generato tramite l'analisi statica di SonarQube

3.5 Containerizzazione e Build Automatica

Al fine di garantire la totale portabilità dell'applicativo e annullare le problematiche legate alle dipendenze ambientali, il back-end e il database relazionale sono stati interamente

containerizzati utilizzando l'ecosistema **Docker**.

3.5.1 Dockerfile: Immagine del Back-end

La creazione dell'immagine del server Node.js è definita da un apposito **Dockerfile**. Per ottimizzare il peso e la sicurezza, si è optato per un'immagine base minimale (**node:20-alpine**). Il file istruisce il demone Docker all'installazione delle dipendenze, alla generazione del client Prisma e alla compilazione da TypeScript a JavaScript prima dell'esposizione della porta 3000.

Listing 3.1: Estratto del Dockerfile del Back-end

```
FROM node:20-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
RUN npx prisma generate
RUN npm run build
EXPOSE 3000
CMD ["npm", "start"]
```

3.5.2 Orchestrazione e Build Automatica (Docker Compose)

Per automatizzare il processo di build e il deploy simultaneo dei macro-componenti, si è implementato un file di orchestrazione **docker-compose.yml**. Questo strumento di *Infrastructure as Code* (IaC) configura una rete virtuale isolata in cui i container comunicano in modo sicuro. È stato inoltre configurato un volume persistente (**pgdata**) per garantire che i record del database non vadano persi alla distruzione dei container.

Listing 3.2: Configurazione di orchestrazione docker-compose.yml

```
version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: bugboard26_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  backend:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://postgres:admin@db:5432/bugboard26_db
    depends_on:
      - db

volumes:
  pgdata:
```

3.5.3 Istruzioni di Deploy e Inizializzazione

Per garantire la totale riproducibilità dell’ambiente da parte di terzi, di seguito vengono documentati i comandi sequenziali necessari per avviare e inizializzare l’infrastruttura di back-end partendo da zero. Tutte le operazioni devono essere eseguite tramite terminale dalla cartella radice del server.

1. **Avvio dell’infrastruttura:** Il seguente comando avvia l’orchestrazione, forzando la *build* dell’immagine Node.js e avviando i container in modalità *detached* (in background).

```
docker-compose up -d --build
```

2. **Sincronizzazione dello Schema (Prisma):** Attendere circa 5-10 secondi affinché il demone PostgreSQL sia completamente inizializzato e pronto ad accettare connessioni. Successivamente, entrare nel container in esecuzione ed eseguire il push dello schema relazionale:

```
docker-compose exec backend npx prisma db push
```

3. **Seeding del Database (RBAC):** Essendo la registrazione pubblica disabilitata per ragioni di sicurezza, è obbligatorio generare l’utente Amministratore di sistema tramite l’apposito script di seed per poter effettuare il primo accesso.

```
docker-compose exec backend npx prisma db seed
```

Al termine di questa sequenza, il sistema è pienamente operativo e la documentazione interattiva delle API risulta accessibile all’indirizzo `http://localhost:3000/api-docs`. Per arrestare l’infrastruttura preservando i dati, è sufficiente lanciare il comando `docker-compose down`.

3.5.4 Cloud Readiness e Predisposizione al Deployment

In linea con i vincoli architetturali e le *best practice* di settore, il sistema è stato ingegnerizzato per essere nativamente *Cloud-Ready*. La separazione netta tra Front-end e Back-end, unita alla containerizzazione integrale del livello server, garantisce che l’applicativo sia agnostico rispetto all’infrastruttura fisica ospitante.

Questa configurazione, guidata dall’approccio *Infrastructure as Code* (IaC) del `docker-compose.yml`, risponde al requisito di auspicabilità relativo al deployment su reti pubbliche. L’intera istanza Back-end è strutturalmente predisposta per il *provisioning* automatizzato su piattaforme di *Public Cloud Computing* (quali Amazon Web Services EC2 o Microsoft Azure), garantendo scalabilità orizzontale e accessibilità degli endpoint REST attraverso la rete Internet per l’eventuale messa in opera del prodotto in produzione.

Capitolo 4

Attività di Testing e Valutazione dell'Usabilità

4.1 Strategia e Adozione dei Pattern xUnit

La validazione logica dell'applicativo è stata affidata a una batteria di test di unità automatici. Per l'ecosistema Node.js/TypeScript è stato adottato il framework **Jest**. Pur non recando il suffisso nominale classico, Jest implementa rigorosamente l'intera architettura formale della famiglia *xUnit*, definendo suite di isolamento, metodi di setup/teardown (es. `beforeEach`) e meccanismi di asserzione avanzati per la verifica degli output.

4.1.1 Ottimizzazione del Compilatore per i Test

Durante la configurazione dell'ambiente di collaudo, è emersa un'incompatibilità strutturale tra le API interne del trasformatore standard (`ts-jest`) e la versione avanzata del compilatore TypeScript adottata (v7.x). Per risolvere questa criticità senza degradare la versione del linguaggio, si è proceduto all'integrazione di `@swc/jest`. Questo compilatore ad alte prestazioni, sviluppato in Rust, ha permesso di svincolare l'esecuzione dei test dalle dipendenze rigide del type-checker, garantendo un'elaborazione istantanea e a prova di errore delle suite.

4.2 Test Plan: Transizione di Stato delle Issue

Prima di procedere con l'implementazione dei test automatici, è stato redatto un Piano di Test formale per definire la strategia di collaudo di una delle funzionalità più critiche del dominio applicativo: l'aggiornamento dello stato di una segnalazione.

Identificativo	TP-01: Validazione Business Logic - Aggiornamento Stato
Obiettivo	Verificare che la logica di transizione di stato di una issue (da TODO a IN_PROGRESS a RESOLVED) sia sicura, tracciata e limitata agli utenti autorizzati.
Elementi da Testare	Metodo <code>updateStatus</code> della classe <code>IssueService</code> .
Ambiente di Collaudo	Framework xUnit (Jest) potenziato con compilatore <code>@swc/jest</code> . Esecuzione in ambiente Node.js disconnesso dal database fisico.
Approccio	Unit Testing in isolamento (White-box/Black-box). Il livello di persistenza (Prisma ORM) viene interamente simulato tramite Mocking per garantire la totale indipendenza e l'alta velocità di esecuzione.
Criteri di Successo	Pass: Il sistema permette il cambio di stato sollevando eccezioni corrette in caso di issue inesistente o permessi insufficienti. Fail: L'operazione va a buon fine bypassando i controlli RBAC o omettendo la registrazione dell'Audit Log.

Tabella 4.1: Documento di Test Plan per la funzionalità di aggiornamento stato

4.3 Progettazione ed Esecuzione dei Test di Unità

L'attività di collaudo si è concentrata sulla validazione isolata della logica di business. Nel rispetto dei requisiti di progetto, sono state sviluppate suite di test automatici per tre metodi non banali appartenenti alla classe `IssueService`: `updateStatus`, `assignUser` e `addTag`.

Per testare i servizi in completo isolamento dal database fisico, si è impiegata la tecnica del *Mocking*. L'oggetto `PrismaClient` è stato fittiziamente sovrascritto, simulando in memoria i comportamenti delle entità e le logiche di transazione atomica (`$transaction`).

4.3.1 Test Metodo 1: updateStatus (IssueService)

Il metodo gestisce la transizione di stato dei bug segnalati. La complessità risiede nel garantire che solo l'assegnatario del task o un amministratore possano effettuare la modifica, oltre a verificare l'esistenza del record.

Copertura delle Classi di Equivalenza (Black-Box)

TC	Scenario / Classe di Equivalenza	Input / Precondizione	Risultato Atteso
TC1	Non Valido (Errore Dominio): Tentativo di aggiornamento su ID inesistente.	Issue = Null	Errore: "Segnalazione non trovata."
TC2	Non Valido (Violazione Permessi): Utente senza privilegi tenta la modifica.	Ruolo = MEMBER Utente $\neq$ Assegnatario	Errore: "Permesso negato..."
TC3	Valido (Happy Path): Utente autorizzato aggiorna un task esistente.	Issue = Trovata Ruolo = MEMBER (Assegnatario)	Aggiornamento eseguito e Transazione invocata.

Tabella 4.2: Derivazione dei Test Case per il metodo updateStatus

Listing 4.1: Test suite per il metodo updateStatus

```
import { IssueService } from '../issueService';
import { prisma } from '../..index';
import { IssueStatus } from '@prisma/client';

// 1. MOCK DI PRISMA: Sostituzione delle chiamate reali con jest.fn()
jest.mock('../..index', () => ({
  prisma: {
    issue: { findUnique: jest.fn(), update: jest.fn() },
    user: { findUnique: jest.fn() },
    historyLog: { create: jest.fn() },
    notification: { create: jest.fn() },
    $transaction: jest.fn(),
  }
})));

describe('IssueService - updateStatus', () => {
  let issueService: IssueService;

  // Pattern xUnit: Teardown e Isolamento
  beforeEach(() => {
    issueService = new IssueService();
    jest.clearAllMocks();
  });

  // TEST 1: Segnalazione inesistente
  it('Dovrebbe lanciare errore se la segnalazione non esiste', async () => {
    (prisma.issue.findUnique as jest.Mock).mockResolvedValue(null);
```



```

    await expect(issueService.updateStatus(999, IssueStatus.IN_PROGRESS,
      1))
      .rejects.toThrow('Segnalazione non trovata.');
```

});

// TEST 2: Controllo Accessi

```

it('Dovrebbe lanciare errore se l\'utente non ha i permessi', async ()
  => {
  (prisma.issue.findUnique as jest.Mock).mockResolvedValue({
    id: 1, status: IssueStatus.TODO, assigneeId: 2
  });
  (prisma.user.findUnique as jest.Mock).mockResolvedValue({
    id: 1, role: 'MEMBER'
  });

  await expect(issueService.updateStatus(1, IssueStatus.IN_PROGRESS, 1))
    .rejects.toThrow('Permesso negato: solo l\'utente assegnato (o un
      Amministratore) può modificare lo stato.');
```

});

// TEST 3: Percorso di Successo (Happy Path)

```

it('Dovrebbe aggiornare lo stato e lanciare la transazione', async () =>
  {
  (prisma.issue.findUnique as jest.Mock).mockResolvedValue({
    id: 1, status: IssueStatus.TODO, assigneeId: 1, reporterId: 3
  });
  (prisma.user.findUnique as jest.Mock).mockResolvedValue({
    id: 1, role: 'MEMBER'
  });

  const mockUpdatedIssue = { id: 1, status: IssueStatus.RESOLVED };
  (prisma.$transaction as
    jest.Mock).mockResolvedValue([mockUpdatedIssue]);

  const result = await issueService.updateStatus(1,
    IssueStatus.RESOLVED, 1);

  expect(result).toEqual(mockUpdatedIssue);
  expect(prisma.$transaction).toHaveBeenCalledTimes(1);
  });
});
```

4.3.2 Test Metodo 2: assignUser (IssueService)

Il metodo `assignUser` gestisce l'assegnazione di un task a un membro del team, richiedendo tre parametri (`issueId`, `assigneeId`, `modifierId`). La complessità risiede nel garantire che l'operazione sia permessa solo agli utenti con privilegi amministrativi.

Copertura delle Classi di Equivalenza (Black-Box)

TC	Scenario / Classe di Equivalenza	Input / Precondizione	Risultato Atteso
TC4	Non Valido (Violazione Permessi): Un membro base tenta di assegnare un task.	Ruolo Modificatore = MEMBER	Errore: "Permesso negato..."
TC5	Valido (Happy Path): Admin assegna un task esistente a un utente valido.	Issue = Trovata Ruolo Modificatore = ADMIN	Assegnazione eseguita e Transazione invocata.

Tabella 4.3: Derivazione dei Test Case per il metodo `assignUser`

Listing 4.2: Test suite per il metodo `assignUser`

```
describe('IssueService - assignUser', () => {
  let issueService: IssueService;

  beforeEach(() => {
    issueService = new IssueService();
    jest.clearAllMocks();
  });

  // TEST 4: Controllo Accessi (Solo Admin)
  it('Dovrebbe lanciare errore se l\'utente non è Admin', async () => {
    (prisma.user.findUnique as jest.Mock).mockResolvedValue({ id: 1, role: 'MEMBER' });

    await expect(issueService.assignUser(1, 2, 1))
      .rejects.toThrow('Permesso negato: solo un Amministratore può assegnare le issue.');
```

```

    expect(result).toEqual(mockUpdatedIssue);
    expect(prisma.$transaction).toHaveBeenCalledTimes(1);
  });
});

```

4.3.3 Test Metodo 3: addTag (IssueService)

Il metodo `addTag` associa un'etichetta semantica a una issue, richiedendo tre parametri (`issueId`, `tagName`, `modifierId`). Il metodo assicura che l'azione sia ristretta al reporter originario, all'assegnatario o a un Amministratore, procedendo con un'operazione *upsert* relazionale.

Copertura delle Classi di Equivalenza (Black-Box)

TC	Scenario / Classe di Equivalenza	Input / Precondizione	Risultato Atteso
TC6	Non Valido (Violazione Permessi): Utente estraneo tenta di aggiungere un tag.	Ruolo = MEMBER Utente $\neq$ Assignee, Reporter	Errore: "Permesso negato..."
TC7	Valido (Happy Path): Aggiunta di un tag da utente autorizzato.	tagName = "frontend" Ruolo = ADMIN	Tag associato tramite operazione di update.

Tabella 4.4: Derivazione dei Test Case per il metodo `addTag`

Listing 4.3: Test suite per il metodo `addTag`

```

describe('IssueService - addTag', () => {
  let issueService: IssueService;

  beforeEach(() => {
    issueService = new IssueService();
    jest.clearAllMocks();
  });

  // TEST 6: Controllo Accessi
  it('Dovrebbe lanciare errore se l\'utente non ha permessi sull\'issue',
    async () => {
      (prisma.issue.findUnique as jest.Mock).mockResolvedValue({
        id: 1, assigneeId: 2, reporterId: 3
      });
      (prisma.user.findUnique as jest.Mock).mockResolvedValue({
        id: 1, role: 'MEMBER'
      });

      await expect(issueService.addTag(1, 'frontend', 1))
        .rejects.toThrow('Permesso negato: solo l\'utente assegnato o un Amministratore può aggiungere etichette.');
```

```
(prisma.user.findUnique as jest.Mock).mockResolvedValue({ id: 1, role:
  'ADMIN' });

const mockIssueWithTag = { id: 1, tags: [{ name: 'frontend' }] };

// addTag utilizza un update diretto, non una transazione
(prisma.issue.update as jest.Mock).mockResolvedValue(mockIssueWithTag);

const result = await issueService.addTag(1, 'frontend', 1);

expect(result).toEqual(mockIssueWithTag);
expect(prisma.issue.update).toHaveBeenCalledTimes(1);
});
});
```

4.4 Valutazione dell'Usabilità

Per garantire che BugBoard26 non sia soltanto funzionalmente corretto, ma anche ergonomico e accessibile, il sistema è stato sottoposto a un duplice processo di validazione dell'usabilità: un'ispezione analitica basata su checklist e un collaudo empirico con utenti reali.

4.4.1 Ispezione Sistematica basata su Checklist (Expert Review)

Prima del rilascio, l'interfaccia utente è stata valutata applicando una checklist di controllo basata sulle euristiche di usabilità consolidate. L'ispezione ha verificato i seguenti parametri critici:

- **Visibilità dello Stato del Sistema:** Verificato. Ogni azione asincrona (es. creazione di una issue o caricamento di una pagina) è accompagnata da feedback visivi immediati (spinner di caricamento e toast notification di successo/errore).
- **Prevenzione dell'Errore:** Verificato. I pulsanti distruttivi (es. archiviazione) o le azioni non permesse (es. cambio di stato per un utente non assegnatario) sono disabilitati a livello visivo (stato *disabled*) e logico.
- **Consistenza e Standard:** Verificato. L'uso delle variabili CSS globali garantisce che la palette cromatica (es. rosso per errori/criticità alta, verde per successo) sia coerente in ogni vista (Board, Modali, Dashboard).
- **Contrasto e Leggibilità:** Verificato. Il contrasto tra il testo primario (`--text-h`) e lo sfondo (`--bg`) rispetta le linee guida WCAG, garantendo un'ottima leggibilità anche su schermi a bassa luminosità.

4.4.2 Test Empirico con Utenti Reali

Per validare l'efficacia del design system, è stato condotto un test di usabilità coinvolgendo un campione eterogeneo di 5 utenti (3 sviluppatori e 2 project manager senza competenze di programmazione). Agli utenti è stato richiesto di completare tre task operativi all'interno dell'applicativo senza alcuna formazione preliminare.

Task Assegnato	Tasso di Successo	Tempo Medio
1. Creare una nuova issue inserendo titolo, tipo e priorità	100% (5/5)	24 secondi
2. Filtrare la Board per visualizzare solo i task "In Progress"	100% (5/5)	12 secondi
3. Aggiornare lo stato di un task e verificare la cronologia	80% (4/5)*	35 secondi

Tabella 4.5: Risultati del collaudo operativo (*1 utente ha richiesto assistenza per individuare il pulsante della cronologia)

I tempi di completamento ridotti e l'elevato tasso di successo hanno confermato l'intuitività dell'interfaccia.

4.4.3 Survey di Valutazione (Questionario)

Al termine della sessione pratica, ai 5 partecipanti è stato somministrato un breve questionario di gradimento strutturato su una scala Likert da 1 (Pessimo/Per nulla) a 5 (Eccellente/Molto).

Domanda del Survey	Punteggio Medio (su 5)
Quanto ritieni intuitiva la navigazione generale del sistema?	4.8 / 5
È stato facile comprendere lo stato e l'assegnatario delle varie issue?	4.6 / 5
Come valuti la leggibilità e la chiarezza visiva della Board principale?	4.8 / 5
I messaggi di errore e le notifiche a schermo sono chiari e utili?	4.4 / 5
Indice di Soddisfazione Complessiva	4.65 / 5

Tabella 4.6: Risultati aggregati del questionario di gradimento post-test

I risultati del survey evidenziano un forte apprezzamento per la chiarezza visiva e l'usabilità generale della piattaforma, validando positivamente le scelte di design intraprese.